

Week 3 Tutorial: Text Files and Strings

This tutorial introduces text files. We will read a short file into a string and do simple string processing.

The central teaching cycle is **predict, run, and modify**.

ILO

- **KU2:** Choose appropriate data types and data structures for different kinds of data.
- **KU3:** Design algorithms to solve simple programming problems.
- **CS1:** Structure small programs using iterations, functions, and files.
- **CS3:** Write readable, descriptive, and well-documented program code.
- **CS9:** Use standard libraries and follow good programming practices.

Objectives

After completing this tutorial, you should be able to:

1. Open a text file for reading.
2. Store the file contents in a string.
3. Remove leading and trailing whitespace with `strip()`.
4. Split a string into lines.
5. Count or search for text in a file.

Activity 1: Create a small text file

Create a file called `message.txt` in the same folder as your Python file. Put these two lines in it:

```
Python is fun.  
Files store text.
```

Activity 2: Read the whole file into a string

```
with open("message.txt", "r") as file:  
    text = file.read()  
  
print(text)
```

The method `read()` returns the complete file as one string. The newline between the two lines is part of the string.

Predict the output of:

```
print(len(text))
print("Python" in text)
print("Java" in text)
```

Activity 3: Remove extra whitespace

```
print(text.strip())
```

`strip()` removes whitespace at the beginning and end of a string. It does not remove the newline between the two lines.

Activity 4: Split the text into lines

```
lines = text.strip().splitlines()

print(lines)
print(len(lines))
```

The result should be:

```
['Python is fun.', 'Files store text.']
2
```

Loop over the lines:

```
for line in lines:
    print(line)
```

Activity 5: Simple string processing

Count the number of words in the file:

```
word_count = 0

for line in lines:
    word_count += len(line.split())

print("Number of words:", word_count)
```

The output is:

```
Number of words: 6
```

Search each line for the word `text`:

```
for line in lines:
    if "text" in line:
        print("Found it:", line)
```

Change the program so that it prints every line in uppercase:

```
for line in lines:
    print(line.upper())
```

Activity 6: Write a string to a file

```
summary = "The file contains " + str(len(lines)) + " lines.\n"

with open("summary.txt", "w") as file:
    file.write(summary)
```

The mode `"w"` means write. It creates the file or replaces its contents.

The mode `"a"` means append:

```
with open("summary.txt", "a") as file:
    file.write("The text was processed by Python.\n")
```

Activity 7: Refactor the functionality

The previous activities put all the code in one place. Now divide the program into small functions. Each function should have one clear responsibility.

Step 1: Read the file

Write a function that opens the file, reads its contents, and returns the text:

```
def read_file(filename):
    with open(filename, "r") as file:
```

```
return file.read()
```

Test it:

```
text = read_file("message.txt")
print(text)
```

Step 2: Count the words

Write a function that receives the text as a parameter and returns the number of words:

```
def count_words(text):
    words = text.split()
    return len(words)
```

Test it:

```
print(count_words(text))
```

Step 3: Find matching lines

Write a function that returns all lines containing a given word:

```
def find_lines(text, word):
    matching_lines = []

    for line in text.splitlines():
        if word in line:
            matching_lines.append(line)

    return matching_lines
```

Test it:

```
for line in find_lines(text, "text"):
    print(line)
```

Step 4: Write a summary

Write a function that saves a summary to a new file:

```
def write_summary(filename, line_count, word_count):  
    with open(filename, "w") as file:  
        file.write(f"Lines: {line_count}\n")  
        file.write(f"Words: {word_count}\n")
```

Step 5: Connect the functions

Put the complete program in a `main()` function. The filename and search word will be supplied as command-line arguments using `sys.argv`:

```
import sys  
  
def main():  
    filename = sys.argv[1]  
    word = sys.argv[2]  
    text = read_file(filename)  
    lines = text.splitlines()  
    words = count_words(text)  
  
    print("Lines:", len(lines))  
    print("Words:", words)  
  
    for line in find_lines(text, word):  
        print("Found:", line)  
  
    write_summary("summary.txt", len(lines), words)  
  
main()
```

Run the program from the terminal like this:

```
python analyse.py message.txt text
```

Here, `sys.argv[0]` is the program name, `sys.argv[1]` is the filename, and `sys.argv[2]` is the word to search for.

The program now has separate functions for reading, counting, searching, and writing. This makes each part easier to test and reuse.

Practice changes

Modify the program so that:

1. The program prints a message when no line contains the search word.

2. The summary file includes the matching lines.
3. The program prints a helpful usage message if the user does not provide both command-line arguments.

Questions

1. What does `file.read()` return?
2. Why do we use `strip()` before `splitlines()`?
3. What is the difference between `"w"` and `"a"`?
4. What does `line.split()` return?
5. What does the expression `"text" in line` check?